



KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

ĐỀ THI KẾT THÚC HỌC PHẦN
MÔN: CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN 2

Đề thi số:
....A...

Họ tên:		MSSV:	
Thời gian:	90 phút	Lớp:	

Câu số 1: DOUBLY LINKED LISTS

Giả sử có 3 lớp đã được khai báo như trong đoạn code sau:

```
1 class Employee
2 {
3     public:
4         string cccd;
5         string ho;
6         string ten;
7
8         Employee(string cc, string h, string t);
9 };
10
11 class Node {
12     public:
13         Employee* data;
14
15         // các con trỏ xuôi, ngược
16         Node* next;
17         Node* previous;
18
19         // hàm khởi tạo
20         Node(Employee* emp);
21 };
22
23 class DBList {
24     private:
25         Node* first;
26         Node* last;
27
28     public:
29         // hàm khởi tạo mặc định
30         DBList();
31
32         // Hàm hủy
33         ~DBList();
34
35         void insertFirst(Employee* e); // chèn 1 nhân viên vào đầu
36         // danh sách
37
38         void insertLast(Employee* e); // chèn 1 nv vào cuối danh sách
```

```

37     Employee* findEmployee(string cccd); // tìm 1 nv có cccd trong
    tham số, không tìm thấy trả về con trỏ NULL
38 };

```

Yêu cầu: Hãy cài đặt 3 phương thức `insertFirst`, `insertLast` và `findEmployee` trong lớp `DBList`.

Câu số 2: BST DELETE NODE

Cho một khai báo lớp `BSTNode` như sau:

```

1 class BSTNode
2 {
3     public:
4         int key;
5         BSTNode *left;
6         BSTNode *right;
7
8         static BSTNode* deleteNode(BSTNode* root, int k);
9
10    public:
11        BSTNode(int newKey);
12 };

```

Yêu cầu: Hãy cài đặt phương thức `static BSTNode* BSTNode::deleteNode(BSTNode* root, int k)` để xóa một node có khóa trong tham số `k` trong cây có gốc ở tham số `root`. Nếu khóa `k` không tồn tại trong cây đó, phương thức này không làm gì cả.

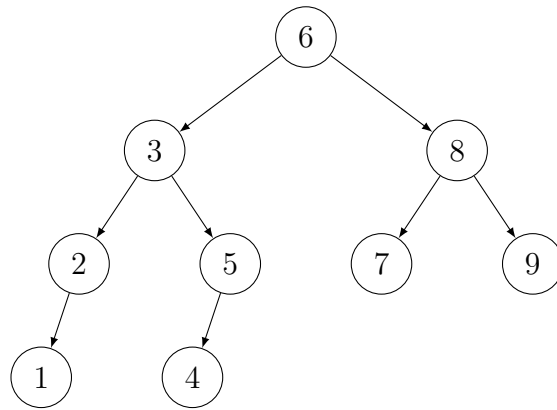
Câu số 3: AVL INSERT (VỚI TRACE LOG)

Giả sử có một chương trình C++ nhận input là một dãy số nguyên, rồi lần lượt chèn từng khóa trong dãy số đó vào 1 cây AVL (lúc đầu là cây rỗng). Trong quá trình thực hiện xây dựng cây, chương trình này sẽ in ra một lịch trình sự kiện (event trace log) như sau:

- Mỗi lần một node mới được tạo ra, chương trình sẽ in ra “`add(k)`” với `k` là khóa của node mới.
- Mỗi lần chương trình quay **phải** tại một node nào đó có khóa là `k`, nó sẽ in ra “`rotateRight(k)`”.
- Mỗi lần chương trình quay **trái** tại một node nào đó có khóa là `k`, nó sẽ in ra “`rotateLeft(k)`”.

VÍ DỤ: Nếu chúng ta nhập vào dãy số: **2, 3, 5, 9, 8, 7, 4, 1** thì chương trình sẽ in trace log sau: `add(2)`, `add(3)`, `add(5)`, `rotateLeft(2)`, `add(6)`, `add(9)`, `rotateLeft(5)`, `add(8)`, `rotateLeft(3)`, `add(7)`, `rotateRight(9)`, `add(4)`, `add(1)`

Và cây AVL kết quả sẽ có dạng như sau:



CÂU HỎI:

Cho dãy số: 14, 17, 11, 7, 53, 4, 13, 12, 8

1. Hãy thực hiện thuật toán chèn cây AVL **bằng tay**, và viết trace log của quá trình **thực hiện từng bước** vào cửa sổ trả lời.
2. Sau đó duyệt **cây kết quả cuối cùng** với thứ tự Post Order (viết một dãy số).

Câu số 4: THUẬT TOÁN DIJKSTRA

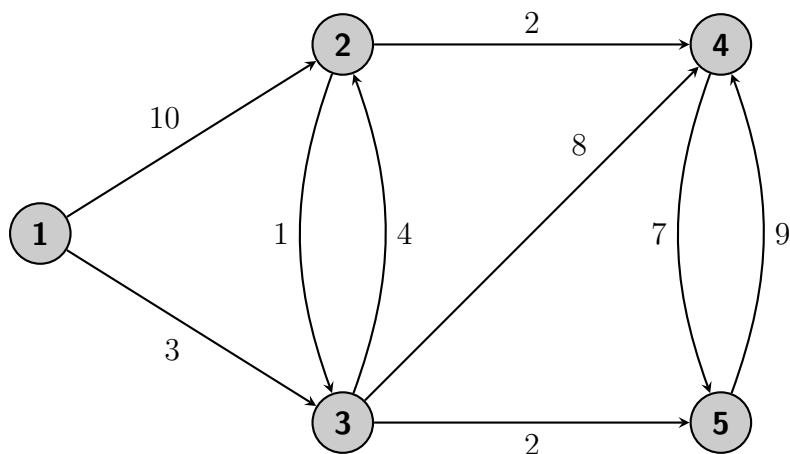
Cho một đồ thị như hình vẽ. Các bạn thực hiện thuật toán Dijkstra theo đúng trình tự từng bước. Trong cửa sổ trả lời mỗi dòng tương ứng với 1 bước theo mẫu sau:

Bước 0: {0, inf, inf, inf, inf} - {F, F, F, F, F}

Bước 1: {0, ..., ..., ..., ...} - {..., ..., ..., ..., ...}

Trong đó:

- Số bước bắt đầu từ 0 (bước khởi tạo).
- Sau số bước là 2 danh sách tương ứng với 2 mảng **dist[]** và **done[]**.
- Giá trị vô tận ký hiệu bằng “inf”.



dist[]						done[]				
	1	2	3	4	5	1	2	3	4	5
	0	∞	∞	∞	∞	F	F	F	F	F

TRƯỞNG BỘ MÔN
(Ký và ghi rõ họ tên)

Hà Nội, ngày 30 tháng 09 năm 2024
GIẢNG VIÊN RA ĐỀ

TS. Trần Khánh Dung

KS. Đoàn Như Tùng



KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

ĐỀ THI KẾT THÚC HỌC PHẦN
MÔN: CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN 2

Đề thi số:
....B...

Họ tên:		MSSV:	
Thời gian:	90 phút	Lớp:	

Câu số 1: DOUBLY LINKED LISTS

Giả sử có 3 lớp đã được khai báo như trong đoạn code sau:

```
1 class Employee
2 {
3     public:
4         string cccd;
5         string ho;
6         string ten;
7
8         Employee(string cc, string h, string t);
9 };
10
11 class Node {
12     public:
13         Employee* data;
14
15         // các con trỏ xuôi, ngược
16         Node* next;
17         Node* previous;
18
19         // hàm khởi tạo
20         Node(Employee* emp);
21 };
22
23 class DBList {
24     private:
25         Node* first;
26         Node* last;
27
28     public:
29         // hàm khởi tạo mặc định
30         DBList();
31
32         // Hàm hủy
33         ~DBList();
34
35         void insertFirst(Employee* e); // chèn 1 nhân viên vào đầu
36         // danh sách
37
38         void insertLast(Employee* e); // chèn 1 nv vào cuối danh sách
```

```

37     Employee* findEmployee(string cccd); // tìm 1 nv có cccd trong
    tham số, không tìm thấy trả về con trỏ NULL
38 };

```

Yêu cầu: Hãy cài đặt 3 phương thức `insertFirst`, `insertLast` và `findEmployee` trong lớp `DBList`.

Câu số 2: BST DELETE NODE

Cho một khai báo lớp `BSTNode` như sau:

```

1  class BSTNode
2  {
3      public:
4          int key;
5          BSTNode *left;
6          BSTNode *right;
7
8          static BSTNode* deleteNode(BSTNode* root, int k);
9
10     public:
11         BSTNode(int newKey);
12 };

```

Yêu cầu: Hãy cài đặt phương thức `static BSTNode* BSTNode::deleteNode(BSTNode* root, int k)` để xóa một node có khóa trong tham số `k` trong cây có gốc ở tham số `root`. Nếu khóa `k` không tồn tại trong cây đó, phương thức này không làm gì cả.

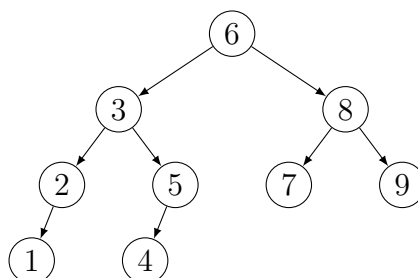
Câu số 3: AVL INSERT (VỚI TRACE LOG)

Giả sử có một chương trình C++ nhận input là một dãy số nguyên, rồi lần lượt chèn từng khóa trong dãy số đó vào 1 cây AVL (lúc đầu là cây rỗng). Trong quá trình thực hiện xây dựng cây, chương trình này sẽ in ra một lịch trình sự kiện (event trace log) như sau:

- Mỗi lần một node mới được tạo ra, chương trình sẽ in ra “`add(k)`” với `k` là khóa của node mới.
- Mỗi lần chương trình quay **phải** tại một node nào đó có khóa là `k`, nó sẽ in ra “`rotateRight(k)`”.
- Mỗi lần chương trình quay **trái** tại một node nào đó có khóa là `k`, nó sẽ in ra “`rotateLeft(k)`”.

VÍ DỤ: Nếu chúng ta nhập vào dãy số: **2, 3, 5, 9, 8, 7, 4, 1** thì chương trình sẽ in trace log sau: `add(2)`, `add(3)`, `add(5)`, `rotateLeft(2)`, `add(6)`, `add(9)`, `rotateLeft(5)`, `add(8)`, `rotateLeft(3)`, `add(7)`, `rotateRight(9)`, `add(4)`, `add(1)`

Và cây AVL kết quả sẽ có dạng như sau:



CÂU HỎI:

Cho dãy số: 50, 25, 10, 5, 7, 3, 30, 20, 8

1. Hãy thực hiện thuật toán chèn cây AVL **bằng tay**, và viết trace log của quá trình **thực hiện từng bước** vào cửa sổ trả lời.
2. Sau đó duyệt **cây kết quả cuối cùng** với thứ tự Post Order (viết một dãy số).

Câu số 4: THUẬT TOÁN DIJKSTRA

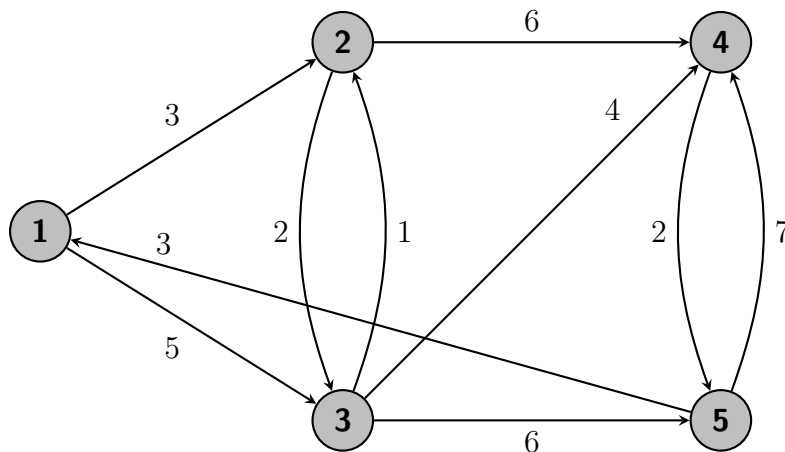
Cho một đồ thị như hình vẽ. Các bạn thực hiện thuật toán Dijkstra theo đúng trình tự từng bước. Trong cửa sổ trả lời mỗi dòng tương ứng với 1 bước theo mẫu sau:

Bước 0: {0, inf, inf, inf, inf} - {F, F, F, F, F}

Bước 1: {0, ..., ..., ..., ...} - {..., ..., ..., ..., ...}

Trong đó:

- Số bước bắt đầu từ 0 (bước khởi tạo).
- Sau số bước là 2 danh sách tương ứng với 2 mảng **dist[]** và **done[]**.
- Giá trị vô tận ký hiệu bằng “inf”.



dist[]						done[]				
	1	2	3	4	5	1	2	3	4	5
	0	∞	∞	∞	∞	F	F	F	F	F

TRƯỞNG BỘ MÔN
(Ký và ghi rõ họ tên)

Hà Nội, ngày 30 tháng 09 năm 2024
GIẢNG VIÊN RA ĐỀ

TS. Trần Khánh Dung

KS. Đoàn Như Tùng